Assignment: Thermophysical Property Data Generation and Toolchain Setup

Learning Objectives

By completing this assignment, students will: 1. Understand the role of **thermophysical property libraries** in engineering simulations. 2. Learn to use the **CoolProp High-Level API** to retrieve saturation and homogeneous phase properties. 3. Configure computational environments in **MATLAB** (GPTIPS2F) and **Julia** (TiSR). 4. Generate a **benchmark thermodynamic dataset** suitable for data-driven modeling. 5. Appreciate the difference between physics-based models and symbolic regression approaches.

Background (Read First)

Modern thermodynamic modeling often follows a three-layer workflow:

1. **Physics-based property evaluation** – Accurate equations of state (EOS) provide reliable data.
2. **Data-driven equation discovery** – Symbolic regression attempts to rediscover governing equations from data.
3. **Physics-informed learning** – Constraints from thermodynamics are enforced during model discovery.

In this assignment: - **CoolProp** is used as the *ground-truth thermodynamic engine*. - **GPTIPS2F (MATLAB)** represents unconstrained symbolic regression. - **TiSR (Julia)** represents physics-informed symbolic regression.

Software and Resources (Access Before Starting)

Students must access and install the following:

1. CoolProp

- Official website: https://coolprop.org
- Documentation (Python): http://www.coolprop.org/coolprop/HighLevelAPI.html

2. Python

- Download: https://www.python.org/downloads/
- Recommended version: Python 3.9 or later

3. MATLAB (for later parts)

- MATLAB product page: https://www.mathworks.com/products/matlab.html
- Required toolboxes:
  - Statistics and Machine Learning Toolbox
  - Global Optimization Toolbox (recommended)

4. GPTIPS2F (MATLAB)

- GPTIPS2 toolbox information: https://sites.google.com/site/gptips4matlab/
- GPTIPS2F distribution (if provided by instructor): ZIP archive

5. Julia

- Download Julia: https://julialang.org/downloads/
- Julia package manager documentation: https://pkgdocs.julialang.org/

6. TiSR (Julia)

- TiSR repository or reference (as provided by instructor)
- Julia symbolic regression backend: https://github.com/MilesCranmer/SymbolicRegression.jl

---

Part A: Understanding CoolProp and Thermodynamic States

Question A1: Saturation vs Homogeneous Phase (Conceptual)

Answer the following in your own words: 1. What is meant by **saturation properties** of a fluid? 2. What is meant by a **homogeneous (single-phase) state**? 3. Give two examples of thermodynamic properties commonly evaluated at saturation. 4. Give two examples of properties evaluated in a homogeneous phase.

(Limit: 150–200 words)

Question A2: CoolProp High-Level API

Study the CoolProp High-Level API documentation and answer: 1. What is the general syntax of the `PropsSI` function? 2. What does the quality parameter `Q` represent? 3. Why does CoolProp always use SI units internally?

Part B: Environment Configuration

Question B1: Python Environment Setup

1. Install Python on your system.

2. Install CoolProp using `pip`.
3. Verify the installation by evaluating the saturation temperature of water at 1 atm.

Deliverable: - Screenshot or text output showing successful execution.

Question B2: MATLAB Path Configuration for GPTIPS2F

1. Download and extract the GPTIPS2F toolbox.
2. Add the toolbox directory to the MATLAB path using `addpath` and `genpath`.
3. Verify installation using the `which gptips2` command.

Deliverable: - MATLAB command window output confirming successful path configuration.

Question B3: Julia Environment Setup for TiSR

1. Install Julia and verify the version.
2. Create a dedicated Julia project environment.
3. Install the following packages:
    - CoolProp
    - SymbolicRegression
    - DataFrames
4. Verify that CoolProp can be called from Julia.

Deliverable: - Julia REPL output showing successful property evaluation.

Part C: Programming Assignment – Benchmark Dataset Generation

Objective

Develop a **Python script** that uses CoolProp to generate a benchmark dataset of temperature (T), pressure (p), and density (ρ).

Fluids to be Studied

1. Methane
2. R134a

Question C1: Define the Thermodynamic Domain

For each fluid: 1. Select a temperature range covering: - Subcooled or compressed region - Superheated vapor region 2. Select a pressure range that avoids two-phase states. 3. Justify your choice of ranges using physical reasoning.

Question C2: Python Script Development

Write a Python script that: 1. Imports the CoolProp library. 2. Creates a grid of temperature and pressure values. 3. Evaluates density (ρ) using the CoolProp High-Level API. 4. Stores the results in a structured format.

Requirements: - Use SI units throughout. - Handle errors when invalid thermodynamic states are encountered.

Question C3: Dataset Export

1. Save the generated dataset as a CSV file for each fluid.
2. The CSV file must contain the following columns:
    - Temperature (K)
    - Pressure (Pa)
    - Density (kg/m^3)

Part D: Reflection and Analysis

Question D1: Physics vs Data

Answer the following: 1. Why is CoolProp considered a physics-based model? 2. Why might a symbolic regression model fail to extrapolate outside the dataset range? 3. How could thermodynamic constraints improve data-driven models?

(Limit: 200–250 words)

Submission Guidelines

- Submit a single ZIP file containing:
    - Python script(s)
    - Generated CSV datasets
    - A PDF or DOCX report answering conceptual questions
- Clearly label all files and figures.

Expected Learning Outcome

After completing this assignment, students should be able to: - Confidently retrieve thermodynamic properties using CoolProp - Set up multi-language scientific computing environments - Generate clean, reproducible datasets for symbolic regression studies - Understand the importance of physics constraints in data-driven modeling

End of Assignment